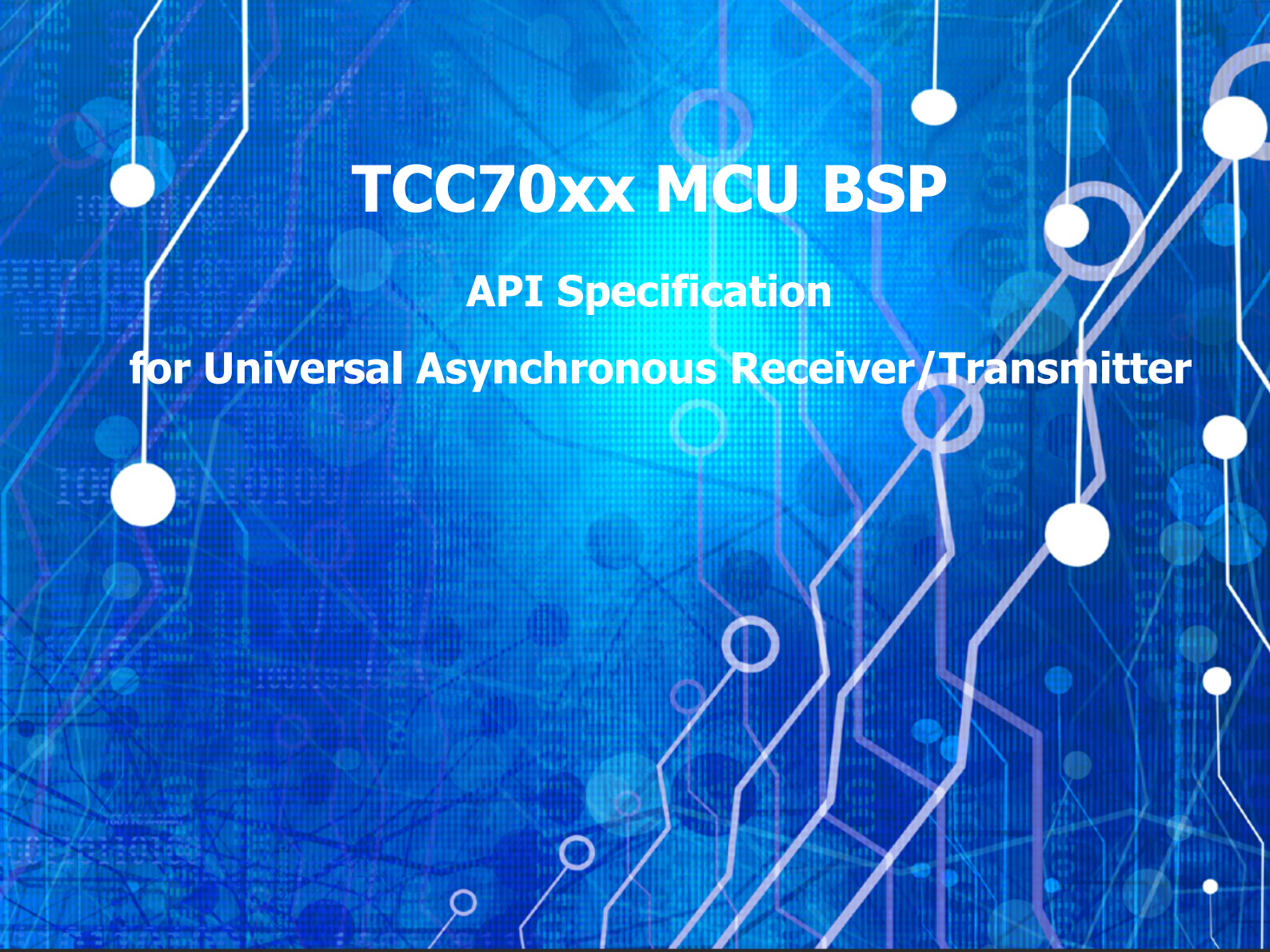




TCC70xx MCU BSP

API Specification

for Universal Asynchronous Receiver/Transmitter

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files	4
4	Constants.....	5
4.1	UART Channel Values.....	5
4.2	UART Communication Method.....	6
4.3	UART Debug Setting Values.....	7
4.4	UART Register Address.....	8
4.5	UART Flag Register Field Values	9
4.6	UART Line Control Register Field Values.....	10
4.7	UART Control Register Field Values.....	11
5	APIs	12
5.1	UART_Init	12
5.2	UART_Open	13
5.3	UART_Read.....	15
5.4	UART_Write	16
5.5	UART_GetData	17
5.6	UART_GetChar	18
5.7	UART_PutChar	19
5.8	UART_SetLineControlReg.....	20
5.9	UART_SetInterruptClearReg.....	21
5.10	UART_SetErrorClearReg	22
5.11	UART_GetReceiveStatusReg	23
5.12	UART_GetRawInterruptStatusReg	24
6	How to use UART Driver	25
6.1	Initialize and Read/Write Data	25
7	References	26
8	Revision History	27
	Rev. 0.10: 2021-09-03	27

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the universal asynchronous receiver/transmitter device driver (hereinafter referred to as UART driver) of TCC70xx MCU BSP. For more detailed information of the universal asynchronous receiver/transmitter in the TCC70xx MCU Sub-System, refer to Chapter 10 Universal Asynchronous Receiver/Transmitter (UART) in "*TCC70xx Full Specification*". [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

UART	Universal Asynchronous Receiver/Transmitter
-------------	---

3 SOURCE FILES

3.1 Location of Source Files

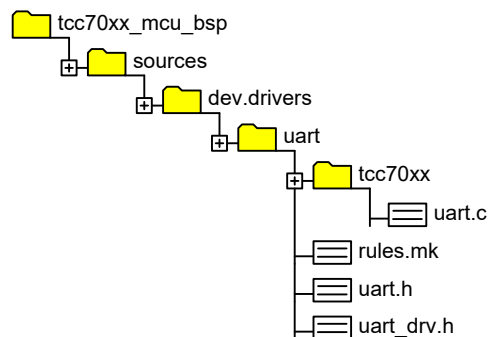


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `uart.c`
 - UART driver source
- `rules.mk`
 - Rules used for build
- `uart.h`
 - UART driver API header
- `uart_drv.h`
 - UART Structures and Defines

4 CONSTANTS

4.1 UART Channel Values

Constant values for UART channel

Declaration

```
#define UART_CH0    (0UL)
#define UART_CH1    (1UL)
#define UART_CH2    (2UL)
#define UART_CH3    (3UL)
#define UART_CH4    (4UL)
#define UART_CH5    (5UL)
#define UART_CH_MAX (6UL)
```

Description

Value	Description
UART_CH0	1st UART Channel
UART_CH1	2nd UART Channel
UART_CH2	3rd UART Channel
UART_CH3	4th UART Channel
UART_CH4	5th UART Channel
UART_CH5	6th UART Channel
UART_CH_MAX	The maximum number of UART channels is 6.

Remark

None

4.2 UART Communication Method

Constant values for UART communication method

Declaration

```
#define UART_POLLING_MODE    (0UL)
#define UART_INTR_MODE      (1UL)
#define UART_DMA_MODE        (2UL)
```

Description

Value	Description
UART_NON_INTR_MODE	The value can be read and written in the UART data register (DR).
UART_INTR_MODE	IRQ Interrupt is generated when reading or writing UART data.
UART_DMA_MODE	UART DMA is generated when reading or writing UART data.

Remark

None

4.3 UART Debug Setting Values

Constant values of UART channel for debugging

Declaration

```
#define UART_DEBUG_CH      (UART_CH0)
#define UART_DEBUG_CLK     (48000000) // 48MHz
```

Description

<i>Value</i>	<i>Description</i>
<i>UART_DEBUG_CH</i>	<i>UART channel used for debugging</i>
<i>UART_DEBUG_CLK</i>	<i>UART clock for debugging</i>

Remark

None

4.4 UART Register Address

Offset values for TCC70xx UART register addresses

Declaration

```
#define UART_REG_DR      (0x00UL)
#define UART_REG_RSR     (0x04UL)
#define UART_REG_ECR     (0x04UL)
#define UART_REG_FR      (0x18UL)
#define UART_REG_ILPR    (0x20UL)
#define UART_REG_IBRD    (0x24UL)
#define UART_REG_FBRD    (0x28UL)
#define UART_REG_LCRH    (0x2cUL)
#define UART_REG_CR      (0x30UL)
#define UART_REG_IFLS    (0x34UL)
#define UART_REG_IMSC    (0x38UL)
#define UART_REG_RIS     (0x3cUL)
#define UART_REG_MIS     (0x40UL)
#define UART_REG_ICR     (0x44UL)
#define UART_REG_DMACR   (0x48UL)
```

Description

Value	Description
UART_REG_DR	Data register
UART_REG_RSR	Receive Status register
UART_REG_ECR	Error Clear register
UART_REG_FR	Flag register
UART_REG_ILPR	IrDA Low-power Counter register
UART_REG_IBRD	Integer Baud rate register
UART_REG_FBRD	Fractional Baud rate register
UART_REG_LCRH	Line Control register
UART_REG_CR	Control register
UART_REG_IFLS	Interrupt FIFO Level status register
UART_REG_IMSC	Interrupt Mask Set/Clear register
UART_REG_RIS	Raw Interrupt Status register
UART_REG_MIS	Masked Interrupt Status register
UART_REG_ICR	Interrupt Clear register
UART_REG_DMACR	DMA Control register

Remark

None

4.5 UART Flag Register Field Values

Field values of TCC70xx UART Flag register

Declaration

```
#define UART_FR_TXFE  (1UL << 7)
#define UART_FR_RXFF  (1UL << 6)
#define UART_FR_TXFF  (1UL << 5)
#define UART_FR_RXFE  (1UL << 4)
#define UART_FR_BUSY  (1UL << 3)
#define UART_FR_CTS   (1UL << 3)
```

Description

Value	Description
UART_FR_TXFE	Transmit FIFO empty
UART_FR_RXFF	Receive FIFO full
UART_FR_TXFF	Transmit FIFO full
UART_FR_RXFE	Receive FIFO empty
UART_FR_BUSY	UART busy
UART_FR_CTS	Clear to send

Remark

None

4.6 UART Line Control Register Field Values

Field values of TCC70xx UART Line Control Register

Declaration

```
#define UART_LCRH_SPS      (1UL << 7)
#define UART_LCRH_WLEN(x) ((x) << 5)
#define UART_LCRH_FEN      (1UL << 4)
#define UART_LCRH_STP2     (1UL << 3)
#define UART_LCRH_EPS      (1UL << 2)
#define UART_LCRH_PEN      (1UL << 1)
#define UART_LCRH_BRK      (1UL << 0)
```

Description

Value	Description
UART_LCRH_SPS	Stick parity
UART_LCRH_WLEN	Word Length
UART_LCRH_FEN	Enable FIFOs
UART_LCRH_STP2	Two stop bits
UART_LCRH_EPS	Even parity
UART_LCRH_PEN	Parity enable
UART_LCRH_BRK	Send break

Remark

None

4.7 UART Control Register Field Values

Field values of TCC70xx UART Control register

Declaration

```
#define UART_CR_CTSEN    (1UL << 15)
#define UART_CR_RTSEN    (1UL << 14)
#define UART_CR_RTS      (1UL << 11)
#define UART_CR_RXE      (1UL << 9)
#define UART_CR_TXE      (1UL << 8)
#define UART_CR_LBE      (1UL << 7)
#define UART_CR_EN       (1UL << 0)
```

Description

Value	Description
UART_CR_CTSEN	CTS hardware flow control enable
UART_CR_RTSEN	RTS hardware flow control enable
UART_CR_RTS	Request to send
UART_CR_RXE	Receive enable
UART_CR_TXE	Transmit enable
UART_CR_LBE	Loopback enable
UART_CR_EN	UART enable

Remark

None

5 APIs

5.1 UART_Init

Initializes the UART channel.

Declaration

```
void UART_Init  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

5.2 UART_Open

Sets all the necessary elements for the UART Channel.

Declaration

```
sint32 UART_Open
(
    uint8      ucCh,
    uint32     uiPriority,
    uint32     uiBaudrate,
    uint8      ucMode,
    uint8      ucCtsRts,
    uint8      ucPortCfg,
    uint8      ucWordLength,
    uint8      ucFIFO,
    uint8      uc2StopBit,
    uint8      ucParity,
    GICIsrFunc fnCallback
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>uiPriority</i>	<i>[in] Interrupt priority (range of parameter: 0 to 15)</i>
<i>uiBaudrate</i>	<i>[in] Baud rate</i>
<i>ucMode</i>	<i>[in] Polling or Interrupt of DMA mode</i>
<i>ucCtsRts</i>	<i>[in] CTS/RTS on/off</i>
<i>ucPortCfg</i>	<i>[in] Port selection</i>
<i>ucWordLength</i>	<i>[in] Word length (5 ~ 6 bits)</i>
<i>ucFIFO</i>	<i>[in] FIFO on/off</i>
<i>uc2StopBit</i>	<i>[in] Two stop bits select</i>
<i>ucParity</i>	<i>[in] Space, even, odd, mark</i>
<i>fnCallback</i>	<i>[in] Callback function</i>

Return

Value	Description
0	Success
-1	Failure

Remark

None

5.3 UART_Read

Receives data as much as the size from the buffer.

Declaration

```
sint32 UART_Read  
(  
    uint8      ucCh,  
    uint8 *    pucBuf,  
    uint32     uiSize  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>pucBuf</i>	<i>[in] Receive buffer</i>
<i>uiSize</i>	<i>[in] Size</i>

Return

Value	Description
<i>Positive number</i>	<i>Data size</i>
<i>-1</i>	<i>Failure</i>

Remark

None

5.4 UART_Write

Writes data to UART data register as much as the size.

Declaration

```
sint32 UART_Write  
(  
    uint8      ucCh,  
    const uint8 * pucBuf,  
    uint32     uiSize  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>pucBuf</i>	<i>[in] Transmit buffer</i>
<i>uiSize</i>	<i>[in] Size</i>

Return

Value	Description
0	Success
-1	Failure

Remark

None

5.5 UART_GetData

Gets data from UART data register.

Declaration

```
uint32 UART_GetData
(
    uint8      ucCh,
    sint32     iWait,
    sint8 *    pcErr
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>iWait</i>	<i>[in] Enable/Disable wait field</i>
<i>pcErr</i>	<i>[out] Error</i>

Return

Value	Description
<i>Positive number</i>	<i>Received data</i>

Remark

If the *iWait* flag is 1, UART_GetData waits until the Receive FIFO becomes empty, and then gets data from the Data Register.

If the *iWait* flag is 0, UART_GetData gets the data from Data Register without a routine of waiting.

5.6 UART_GetChar

Gets character data from UART data register.

Declaration

```
sint32 UART_GetChar
(
    uint8      ucCh,
    sint32     iWait,
    sint8 *    pcErr
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>iWait</i>	<i>[in] Enable/Disable wait field</i>
<i>pcErr</i>	<i>[out] Error</i>

Return

Value	Description
<i>Positive number</i>	<i>Received data</i>
<i>-1</i>	<i>Failure</i>

Remark

If the *iWait* flag is 1, UART_GetChar waits until the Receive FIFO becomes empty, and then gets data from the Data Register.

If the *iWait* flag is 0, UART_GetChar gets the data from the Data Register without a routine of waiting.

5.7 UART_PutChar

Puts character data in UART data register.

Declaration

```
sint32 UART_PutChar  
(  
    uint8  ucCh,  
    uint8  ucChar  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>ucChar</i>	<i>[in] Character data</i>

Return

Value	Description
0	Success
-1	Failure

Remark

None

5.8 UART_SetLineControlReg

Sets the UART Line Control Register (LCR_H).

Declaration

```
void UART_SetLineControlReg
(
    uint8  ucCh,
    uint32 uiBits,
    uint8  uiEnabled
);
```

Parameters

Value	Description
ucCh	[in] UART channel
uiBits	[in] Bit field
uiEnabled	[in] Enable/Disable bit field

Return

None

Remark

None

5.9 UART_SetInterruptClearReg

Clears the error interrupt status in Interrupt Clear Register (ICR).

Declaration

```
void UART_SetInterruptClearReg  
(  
    uint8  ucCh,  
    uint32 uiSetValue  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>uiSetValue</i>	<i>[in] Corresponding error interrupt</i>

Return

None

Remark

None

5.10 UART_SetErrorClearReg

Clears the error status in Error Clear Register (ECR).

Declaration

```
void UART_SetErrorClearReg  
(  
    uint8  ucCh,  
    uint32 uiSetValue  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>
<i>uiSetValue</i>	<i>[in] Corresponding error field</i>

Return

None

Remark

None

5.11 UART_GetReceiveStatusReg

Gets the error status in Receive Status Register (RSR).

Declaration

```
uint32 UART_GetReceiveStatusReg  
(  
    uint8 ucCh  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>

Return

Value	Description
0	No error
0x1 ~ 0x7FF	Corresponding error field

Remark

None

5.12 UART_GetRawInterruptStatusReg

Gets the error interrupt status in Raw Interrupt Status Register (RIS).

Declaration

```
uint32 UART_GetRawInterruptStatusReg  
(  
    uint8 ucCh  
)
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] UART channel</i>

Return

Value	Description
0	No error
0x1 ~ 0x7FF	Corresponding error interrupt field

Remark

None

6 HOW TO USE UART DRIVER

UART driver must use *UART_Open()* function to set UART channel, baud rate, and mode. When reading data, *UART_GetChar()* or *UART_Read()* function is used. When writing data, *UART_Write()* function is used.

6.1 Initialize and Read/Write Data

```
// default settings
```

```
// Default Settings for uart channel 0
#define UART_DEBUG_CH    (UART_CH0)
#define UART_DEBUG_CLK    (48000000) // 48MHz
```

```
void UART_Init
(
    void
)
{
    // For LOG output
    (void)UART_Close(UART_DEBUG_CH);
    (void)UART_Open(UART_DEBUG_CH, GIC_PRIORITY_NO_MEAN, 115200UL, UART_POLLING_MODE,
                    UART_CTSRTS_OFF, 4U, WORD_LEN_8, DISABLE_FIFO, TWO_STOP_BIT_OFF, PARITY_SPACE,
                    NULL_PTR);
}

int main()
{
    sint8  error;
    sint32 ret    = 0;
    uint8  c      = 0;

    UART_Init(); //UART initialization

    for( ; ; )
    {
        ret = UART_GetChar(UART_DEBUG_CH, Disable, (sint8 *)&error); //Read Character data

        if(ret > 0)
        {
            c = ((uint8)ret & 0xFFUL);
            UART_Write(UART_DEBUG_CH, &c, 1UL) //Write 1byte character data
        }
    }

    Return 0;
}
```

7 REFERENCES

- [1] TCC70xx Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

8 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."